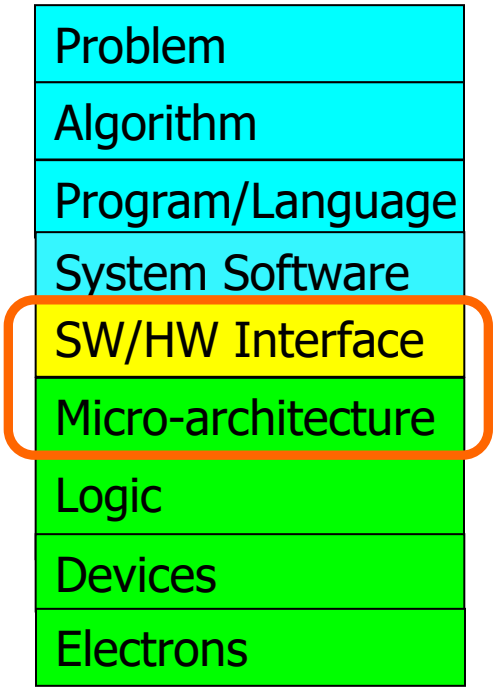
Computer Arch.

Lecture 9-10: Instruction Set Architectures II

Adopted from Prof. Onur Mutlu

Agenda for Today & Next Few Lectures

- The von Neumann model
- LC-3: An example of von Neumann machine
- LC-3 and MIPS Instruction Set Architectures
- LC-3 and MIPS assembly and programming
- Introduction to microarchitecture and single-cycle microarchitecture
- Multi-cycle microarchitecture



Instruction (Processing) Cycle

LC-3: A von Neumann Machine

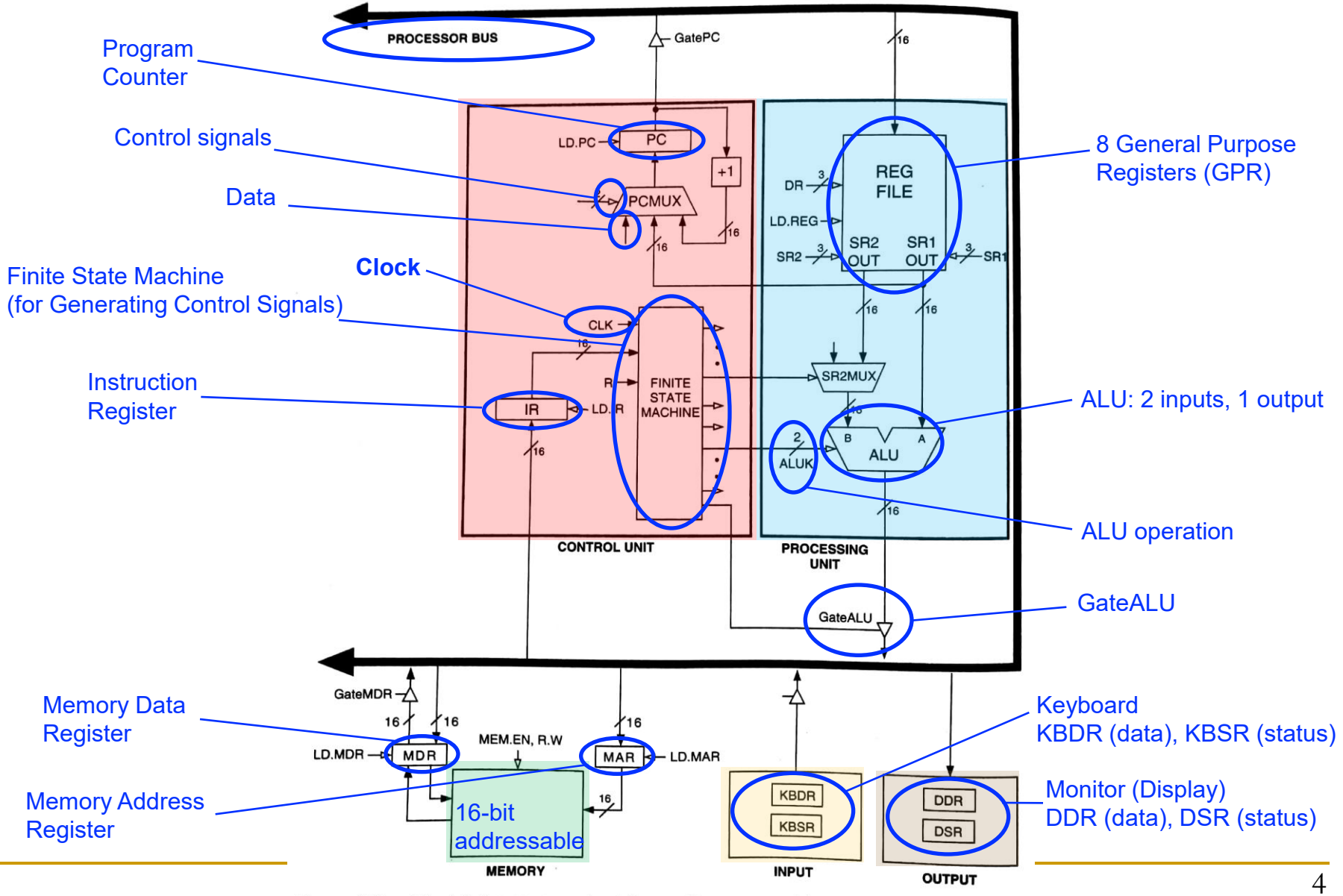


Figure 4.3 The LC-3 as an example of the von Neumann model

Control of the Instruction Cycle

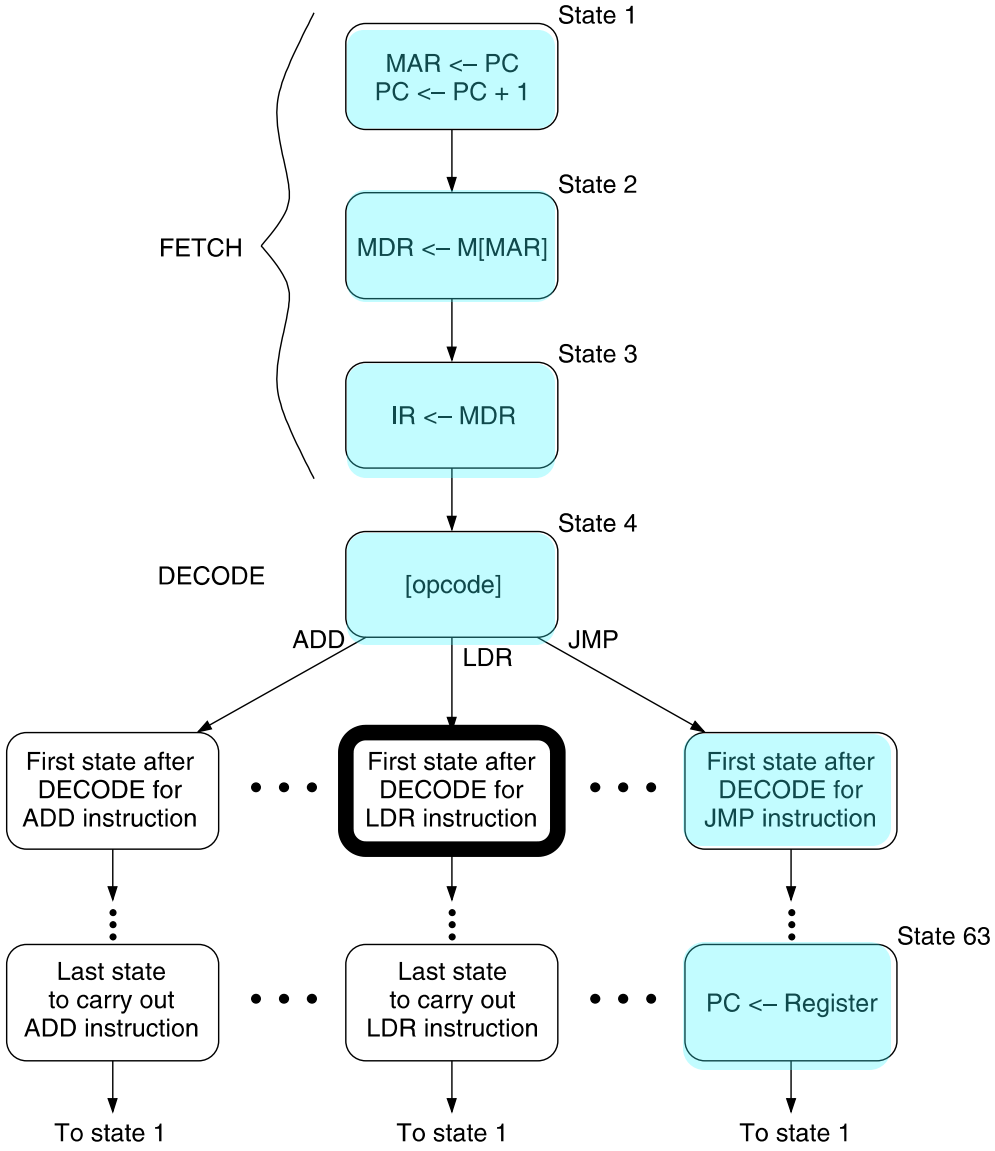


Figure 4.4 An abbreviated state diagram of the LC-3

- State 1
  - The FSM asserts GatePC and LD.MAR
  - It selects input (+1) in PCMUX and asserts LD.PC
- State 2
  - MDR is loaded with the instruction
- State 3
  - The FSM asserts GateMDR and LD.IR
- State 4
  - The FSM goes to next state depending on opcode
- State 63
  - JMP loads register into PC
- Full state diagram in Patt&Pattel, Appendix C

This is an FSM Controlling the LC-3 Processor

To Come: Full State Machine for LC-3b

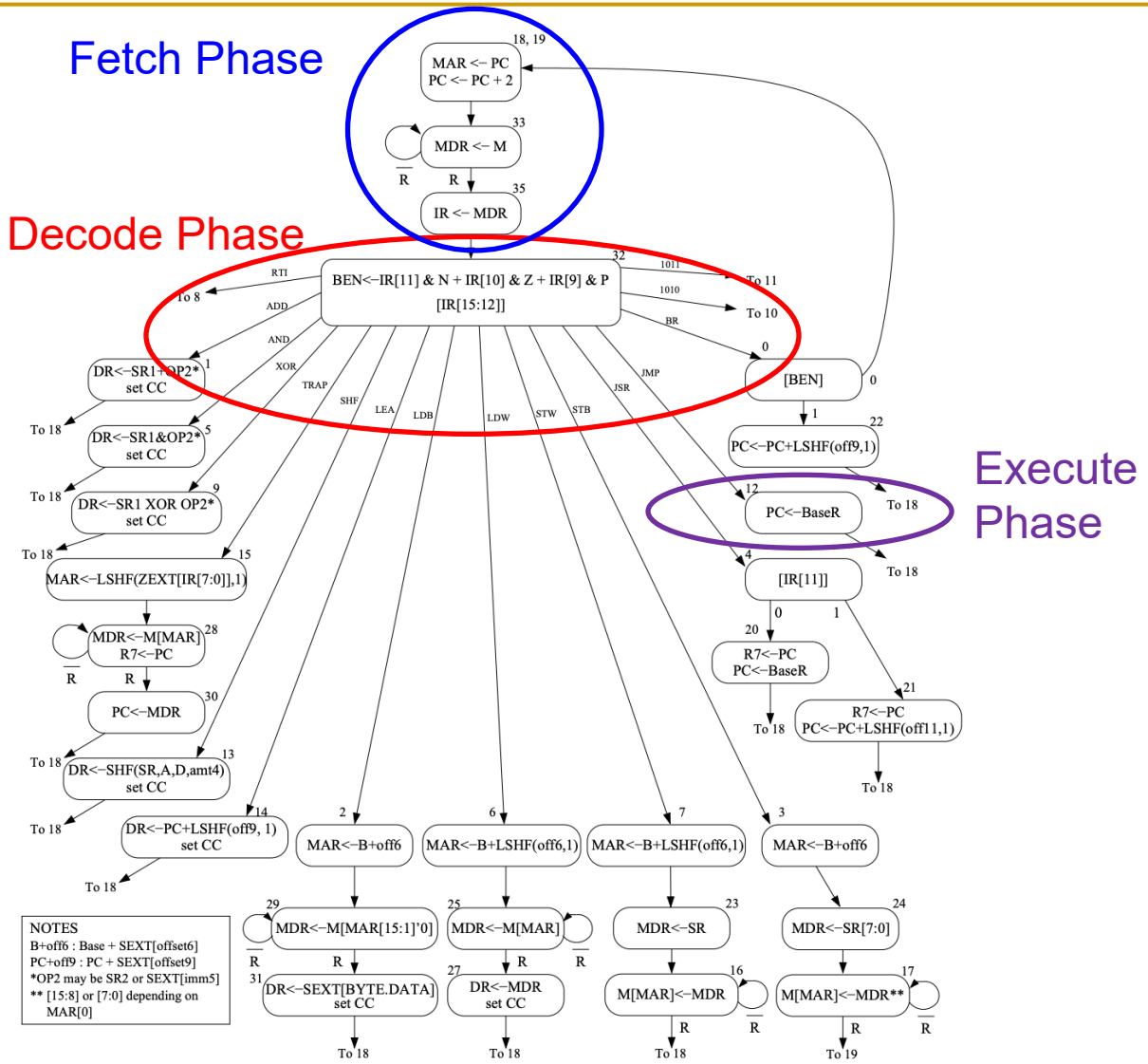


Figure C.2: A state machine for the LC-3b

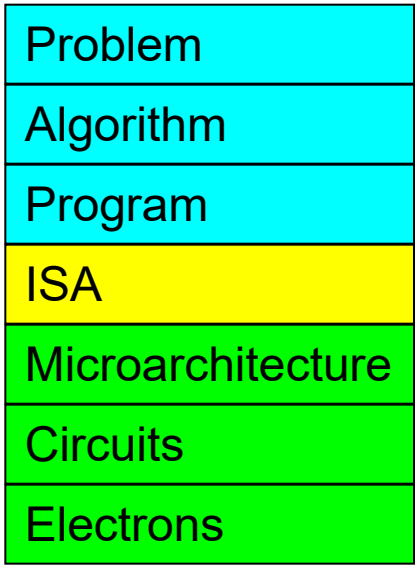
https://safari.ethz.ch/digitaltechnik/spring2022/lib/exe/fetch.php?media=pp-appendixc.pdf

LC-3 and MIPS Instruction Set Architectures

The Instruction Set Architecture

- The ISA is the interface between what the software commands and what the hardware carries out

- The ISA specifies
  - The memory organization
    - Address space (LC-3: 2^{16}, MIPS: 2^{32})
    - Addressability (LC-3: 16 bits, MIPS: 8 bits)
      - Word- or Byte-addressable
  - The register set
    - 8 registers (R0 to R7) in LC-3
    - 32 registers in MIPS
  - The instruction set
    - Opcodes
    - Data types
    - Addressing modes
    - Length and format of instructions



Instructions (Opcodes)

Opcodes

- A large or small set of opcodes could be defined
  - E.g, HP Precision Architecture: an instruction for A*B+C
  - E.g, x86 ISA: multimedia extensions (MMX), later SSE, AVX, AMX
  - E.g, VAX ISA: opcode to save all information of one program prior to switching to another program

- Tradeoffs are involved. Examples:
  - Hardware complexity vs. software complexity
  - Latency of simple vs. complex instructions

- In LC-3 and in MIPS there are three types of opcodes
  - Operate
  - Data movement
  - Control

Opcodes in LC-3

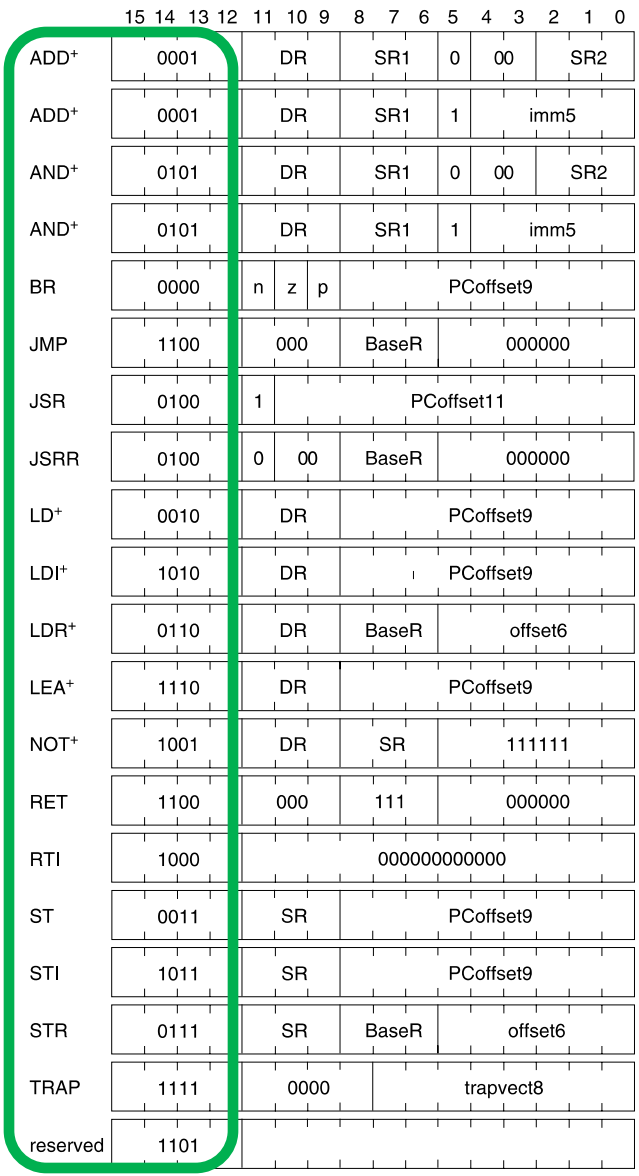
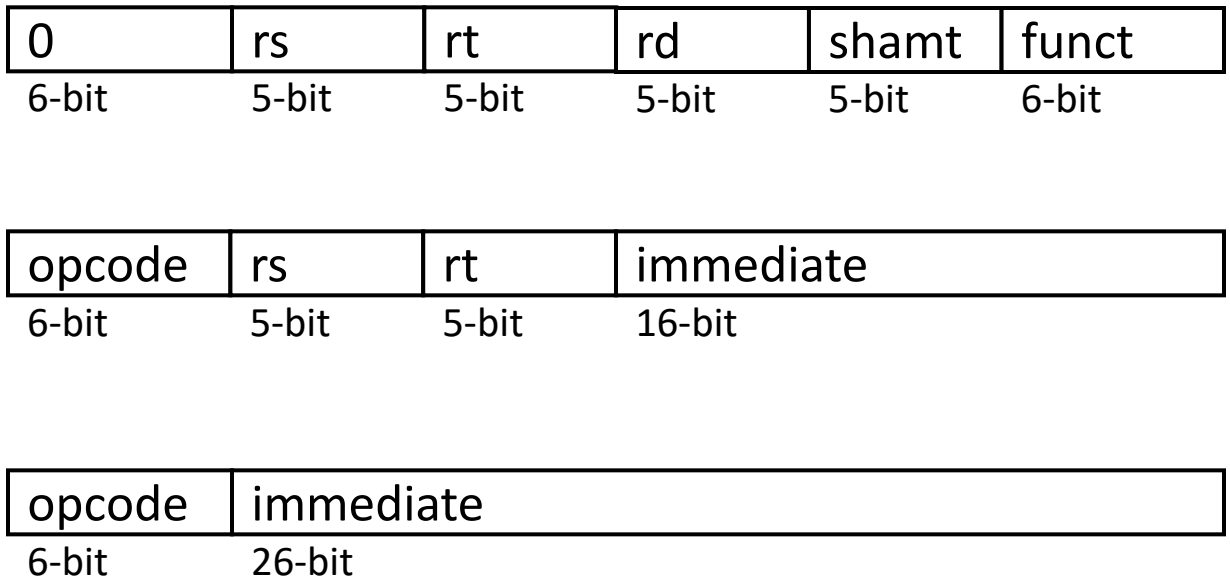


Figure 5.3 Formats of the entire LC-3 instruction set. NOTE: + indicates instructions that modify condition codes

MIPS Instruction Types



Funct in MIPS R-Type Instructions (I)

Opcode is 0 in MIPS R-Type instructions.

Funct defines the operation

Table B.2 R-type instructions, sorted by funct field

Funct	Name	Description	Operation
000000 (0)	`sll rd, rt, shamt`	shift left logical	`[rd] = [rt] << shamt`
000010 (2)	`srl rd, rt, shamt`	shift right logical	`[rd] = [rt] >> shamt`
000011 (3)	`sra rd, rt, shamt`	shift right arithmetic	`[rd] = [rt] >>> shamt`
000100 (4)	`sllv rd, rt, rs`	shift left logical variable	`[rd] = [rt] << [rs]`$_{4:0}$
000110 (6)	`srlv rd, rt, rs`	shift right logical variable	`[rd] = [rt] >> [rs]`$_{4:0}$
000111 (7)	`srav rd, rt, rs`	shift right arithmetic variable	`[rd] = [rt] >>> [rs]`$_{4:0}$
001000 (8)	`jr rs`	jump register	`PC = [rs]`
001001 (9)	`jalr rs`	jump and link register	`$ra = PC + 4, PC = [rs]`
001100 (12)	`syscall`	system call	`system call exception`
001101 (13)	`break`	break	`break exception`
010000 (16)	`mfhi rd`	move from hi	`[rd] = [hi]`
010001 (17)	`mthi rs`	move to hi	`[hi] = [rs]`
010010 (18)	`mflo rd`	move from lo	`[rd] = [lo]`
010011 (19)	`mtlo rs`	move to lo	`[lo] = [rs]`
011000 (24)	`mult rs, rt`	multiply	`{[hi], [lo]} = [rs] × [rt]`
011001 (25)	`multu rs, rt`	multiply unsigned	`{[hi], [lo]} = [rs] × [rt]`
011010 (26)	`div rs, rt`	divide	`[lo] = [rs]/[rt], [hi] = [rs]%[rt]`
011011 (27)	`divu rs, rt`	divide unsigned	`[lo] = [rs]/[rt], [hi] = [rs]%[rt]`

(*continued*)

Funct in MIPS R-Type Instructions (II)

Table B.2 R-type instructions, sorted by funct field—Cont'd

Funct	Name	Description	Operation
100000 (32)	`add rd, rs, rt`	add	`[rd] = [rs] + [rt]`
100001 (33)	`addu rd, rs, rt`	add unsigned	`[rd] = [rs] + [rt]`
100010 (34)	`sub rd, rs, rt`	subtract	`[rd] = [rs] - [rt]`
100011 (35)	`subu rd, rs, rt`	subtract unsigned	`[rd] = [rs] - [rt]`
100100 (36)	`and rd, rs, rt`	and	`[rd] = [rs] & [rt]`
100101 (37)	`or rd, rs, rt`	or	`[rd] = [rs] | [rt]`
100110 (38)	`xor rd, rs, rt`	xor	`[rd] = [rs] ^ [rt]`
100111 (39)	`nor rd, rs, rt`	nor	`[rd] = ~([rs] | [rt])`
101010 (42)	`slt rd, rs, rt`	set less than	`[rs] < [rt] ? [rd] = 1 : [rd] = 0`
101011 (43)	`sltu rd, rs, rt`	set less than unsigned	`[rs] < [rt] ? [rd] = 1 : [rd] = 0`

- More complete list of instructions are in H&H Appendix B

Data Types

Data Types

- An ISA supports one or several data types

- LC-3 only supports 2's complement integers **H&H Chapter 1.4.6**
  - Negative of a 2's complement binary value X = NOT(X) + 1

- MIPS supports
  - 2's complement integers
  - Unsigned integers
  - Floating point

- Tradeoffs are involved. Examples:
  - Hardware complexity vs. software complexity

Complex vs. Simple Instructions+Data Types

- Complex instruction: An instruction does a lot of work, e.g. many operations
  - Compute FFT
  - String copy
  - Matrix multiply
  - …
- Simple instruction: An instruction does little work -- it is a primitive using which complex operations can be built
  - Add
  - XOR
  - Multiply
  - …

Complex vs. Simple Instructions+Data Types

- Advantages of Complex Instructions + Data Types

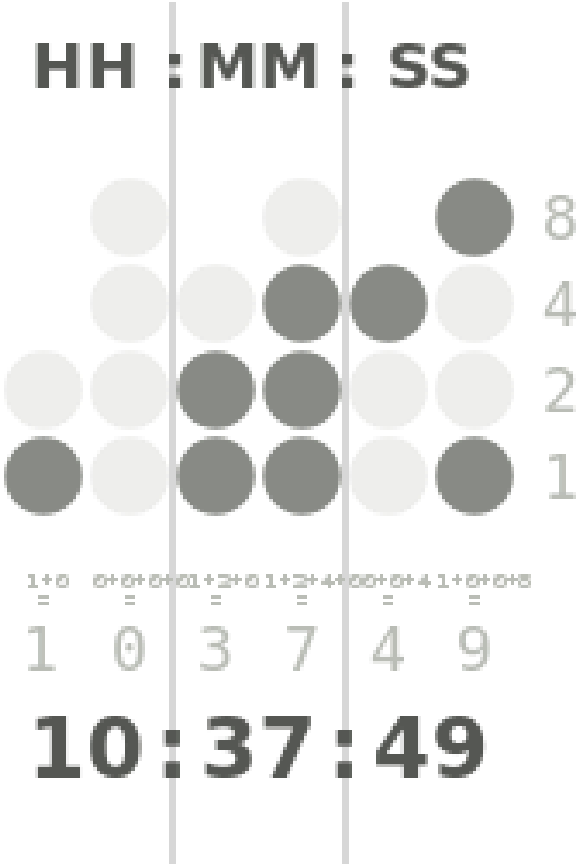
  + Denser encoding → smaller code size → better memory utilization, saves off-chip bandwidth, better cache hit rate

  + Simpler compiler: no need to optimize small instructions as much

- Disadvantages of Complex Instructions + Data Types
  - Larger chunks of work → compiler has less opportunity to optimize
  - More complex hardware → translation from a high level to control signals and optimization needs to be done by hardware

Aside: An Example: **B**inary**C**oded**D**ecimal

- Each decimal digit is encoded with a fixed number of bits



"Binary clock" by Alexander Jones & Eric Pierce - Own work, based on Wapcaplet's Binary clock.png on the English Wikipedia. Licensed under CC BY-SA 3.0 via Wikimedia Commons - http://commons.wikimedia.org/wiki/File:Binary_clock.svg#mediaviewer/File:Binary_clock.svg

"Digital-BCD-clock" by Julo - Own work. Licensed under Public Domain via Wikimedia Commons - http://commons.wikimedia.org/wiki/File:Digital-BCD-clock.jpg#mediaviewer/File:Digital-BCD-clock.jpg

Aside: An Example: **B**inary**C**oded**D**ecimal

- Each decimal digit is encoded with a fixed number of bits



"Binary clock" by Alexander Jones & Eric Pierce - Own work, based on Wapcaplet's Binary clock.png on the English Wikipedia. Licensed under CC BY-SA 3.0 via Wikimedia Commons - http://commons.wikimedia.org/wiki/File:Binary_clock.svg#mediaviewer/File:Binary_clock.svg

"Digital-BCD-clock" by Julo - Own work. Licensed under Public Domain via Wikimedia Commons - http://commons.wikimedia.org/wiki/File:Digital-BCD-clock.jpg#mediaviewer/File:Digital-BCD-clock.jpg

Addressing Modes

Addressing Modes

- An addressing mode is a mechanism for specifying where an operand is located

- There are five addressing modes in LC-3
  - Immediate or literal (constant)
    - The operand is in some bits of the instruction
  - Register
    - The operand is in one of R0 to R7 registers
  - Three memory addressing modes
    - PC-relative
    - Indirect
    - Base+offset

- MIPS has pseudo-direct addressing (for j and jal), additionally, but does **not** have indirect addressing

Data Movement Instructions and Addressing Modes

Data Movement Instructions

- In LC-3, there are seven data movement instructions
  - LD, LDR, LDI, LEA, ST, STR, STI

- Format of load and store instructions
  - Opcode (bits [15:12])
  - DR or SR (bits [11:9])
  - Address generation bits (bits [8:0])
  - Four ways to interpret bits, called addressing modes
    - PC-Relative Mode
    - Indirect Mode
    - Base+Offset Mode
    - Immediate Mode

- In MIPS, there are only Base+offset and Immediate modes for load and store instructions

Sign extension

- ***Refers to the process of increasing the number of bits in a binary number while preserving its sign (positive or negative) and value.***

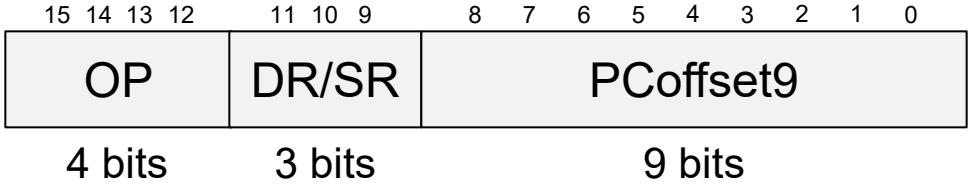
- **Purpose:** Sign extension ensures that when a number is represented with more bits (e.g., moving from 8-bit to 16-bit), its original value and sign are maintained.

- **Method:** It involves replicating the most significant bit (MSB), which represents the sign (0 for positive, 1 for negative), to the left (most significant side) of the number.

- **Example:** If you have a 4-bit two's complement representation of 3 (0011) and want to extend it to 8 bits, you would add four more bits to the left, resulting in 00000011. Similarly, -3 (1101) extended to 8 bits becomes 11111101.

PC-Relative Addressing Mode

- LD (Load) and ST (Store)



  - OP = opcode
    - E.g., LD = 0010
    - E.g., ST = 0011
  - DR = destination register in LD
  - SR = source register in ST
  - LD: DR ← Memory[PC† + sign-extend(PCoffset9)]
  - ST: Memory[PC† + sign-extend(PCoffset9)] ← SR

The "LD r2, offset" instruction, common in assembly languages, loads a value from memory into register r2. The "offset" part specifies the memory address from which the value is loaded. This address is calculated by adding an offset to a base address, which can be another register or a memory location.

† This is the incremented PC

LD in LC-3

- LD assembly and machine code

LC-3 assembly

```
LD R2, 0x1AF
```

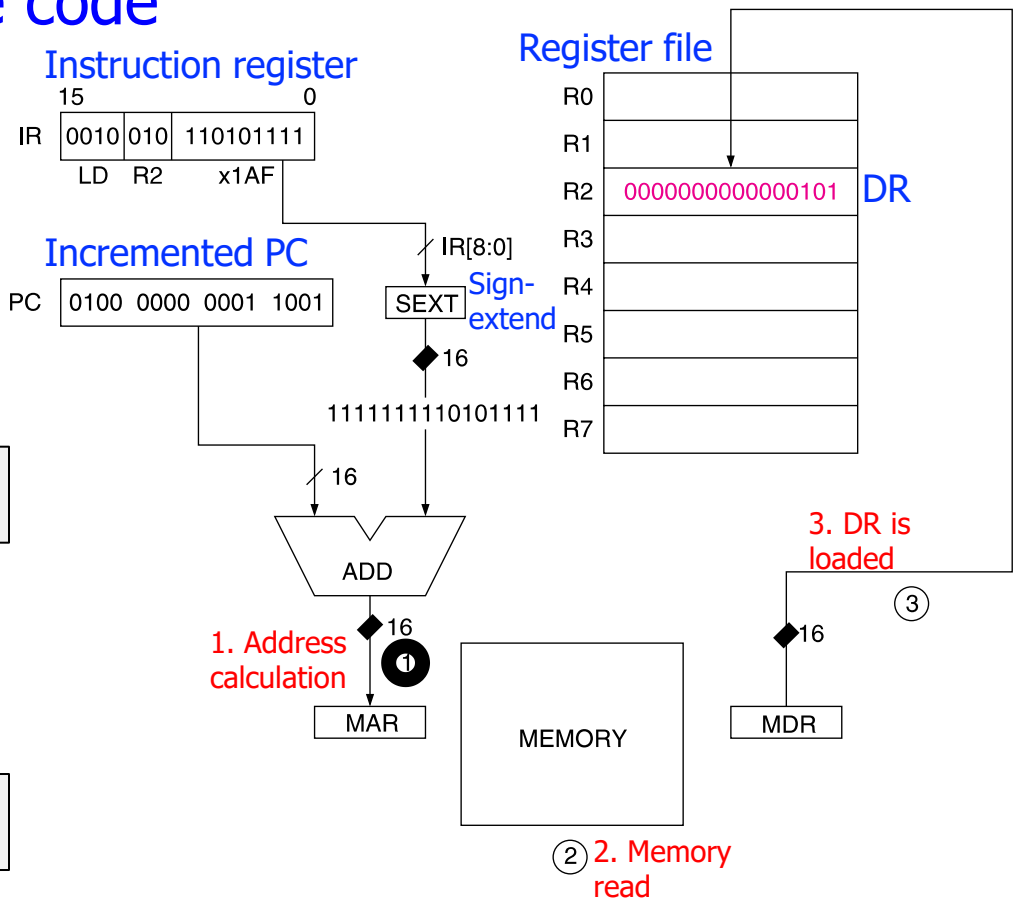
Field Values

OP	DR	PCoffset9
2	2	0x1AF

Machine Code

OP	DR	PCoffset9
0 0 1 0	0 1 0	1 1 0 1 0 1 1 1 1

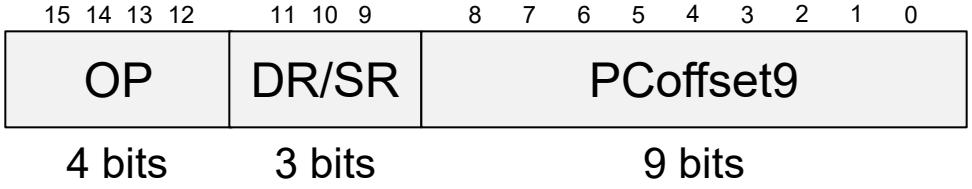
15 12 11 9 8 0



The memory address is only +255 to -256 locations away of the LD or ST instruction

Indirect Addressing Mode

- LDI (Load Indirect) and STI (Store Indirect)



  - OP = opcode
    - E.g., LDI = 1010
    - E.g., STI = 1011
  - DR = destination register in LDI
  - SR = source register in STI
  - LDI: DR ← Memory[Memory[PC† + sign-extend(PCoffset9)]]
  - STI: Memory[Memory[PC† + sign-extend(PCoffset9)]] ← SR

† This is the incremented PC

LDI in LC-3

- LDI assembly and machine code

LC-3 assembly

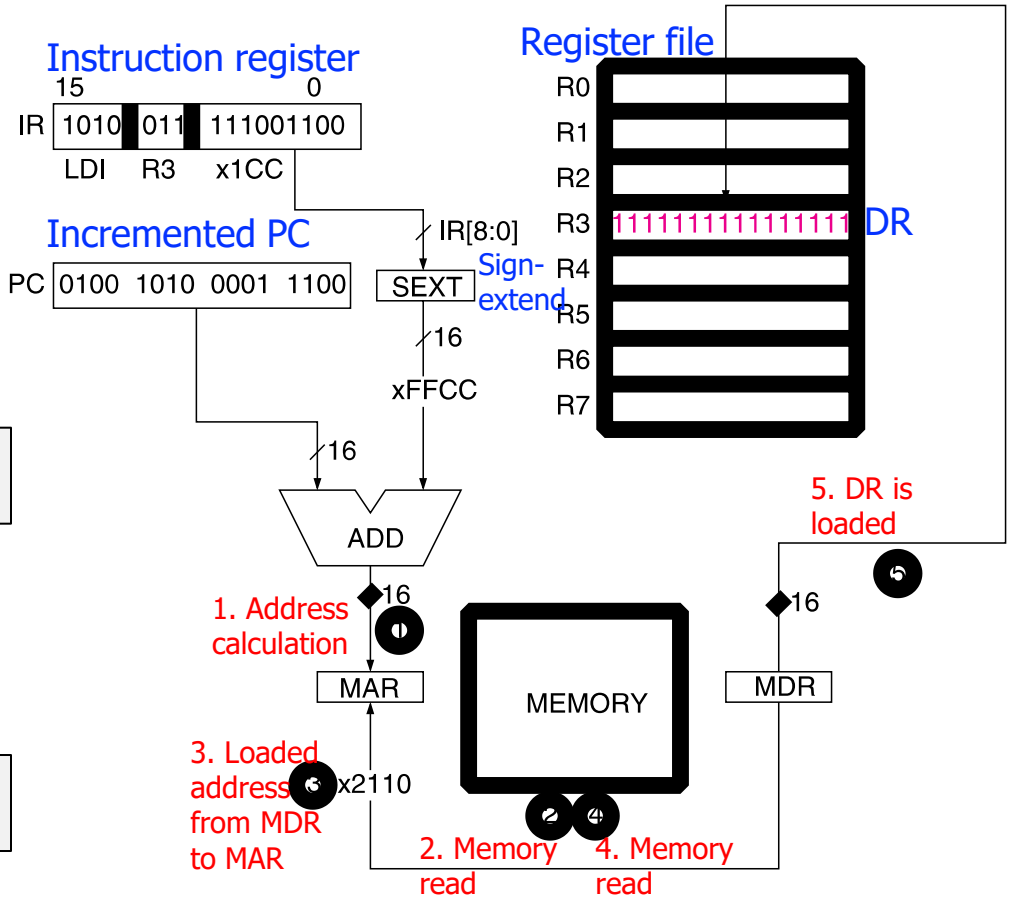
```
LDI R3, 0x1CC
```

Field Values

OP	DR	PCoffset9
A	3	0x1CC

Machine Code

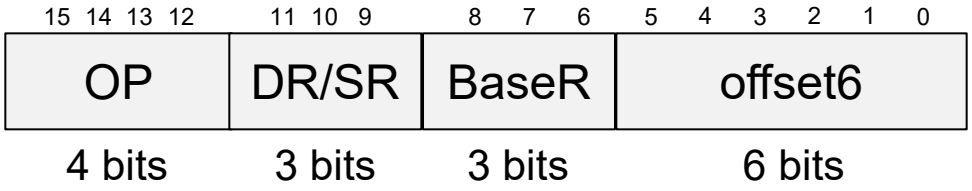
OP	DR	PCoffset9
1 0 1 0	0 1 1	1 1 1 0 0 1 1 0 0
15 12	11 9	8 0



Now the address of the operand can be anywhere in the memory

Base+Offset Addressing Mode

- LDR (Load Register) and STR (Store Register)



  - OP = opcode
    - E.g., LDR = 0110
    - E.g., STR = 0111
  - DR = destination register in LDR
  - SR = source register in STR
  - LDR: DR ← Memory[BaseR + sign-extend(offset6)]
  - STR: Memory[BaseR + sign-extend(offset6)] ← SR

LDR in LC-3

- LDR assembly and machine code

LC-3 assembly

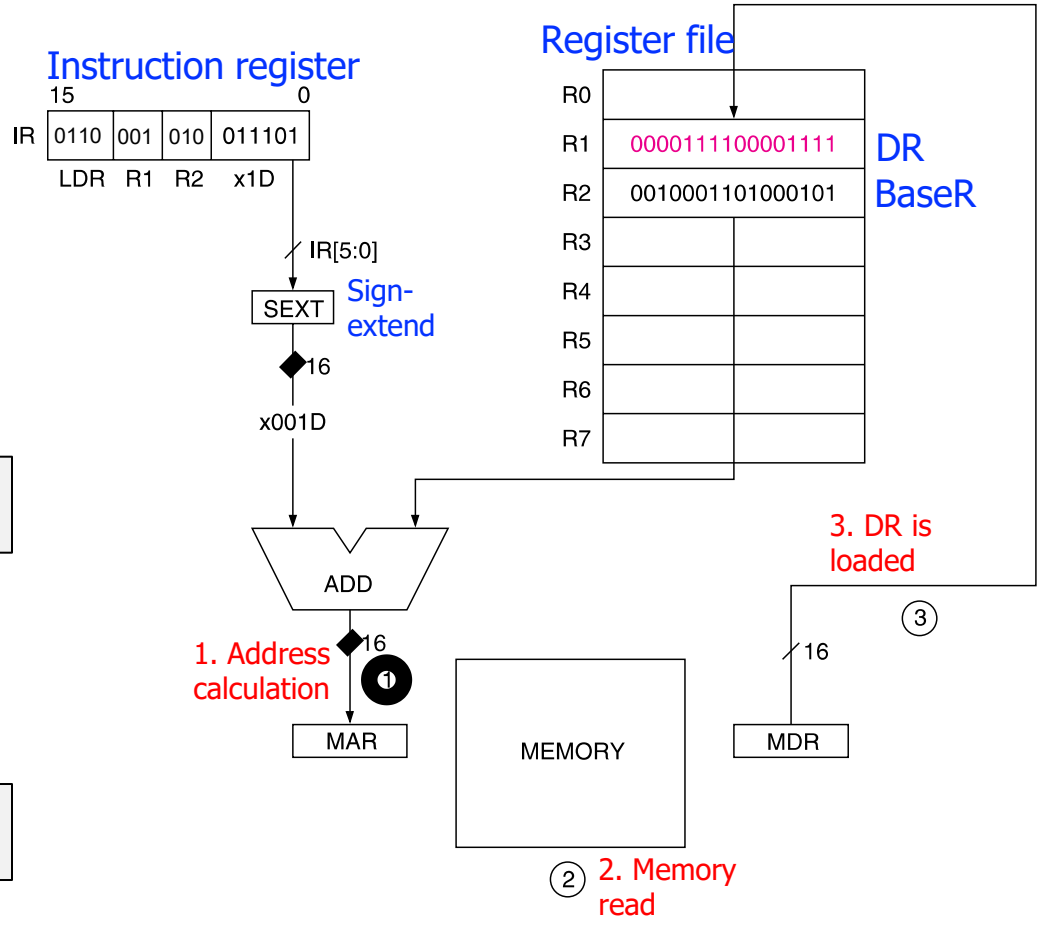
```
LDR R1, R2, 0x1D
```

Field Values

OP	DR	BaseR	offset6
6	1	2	0x1D

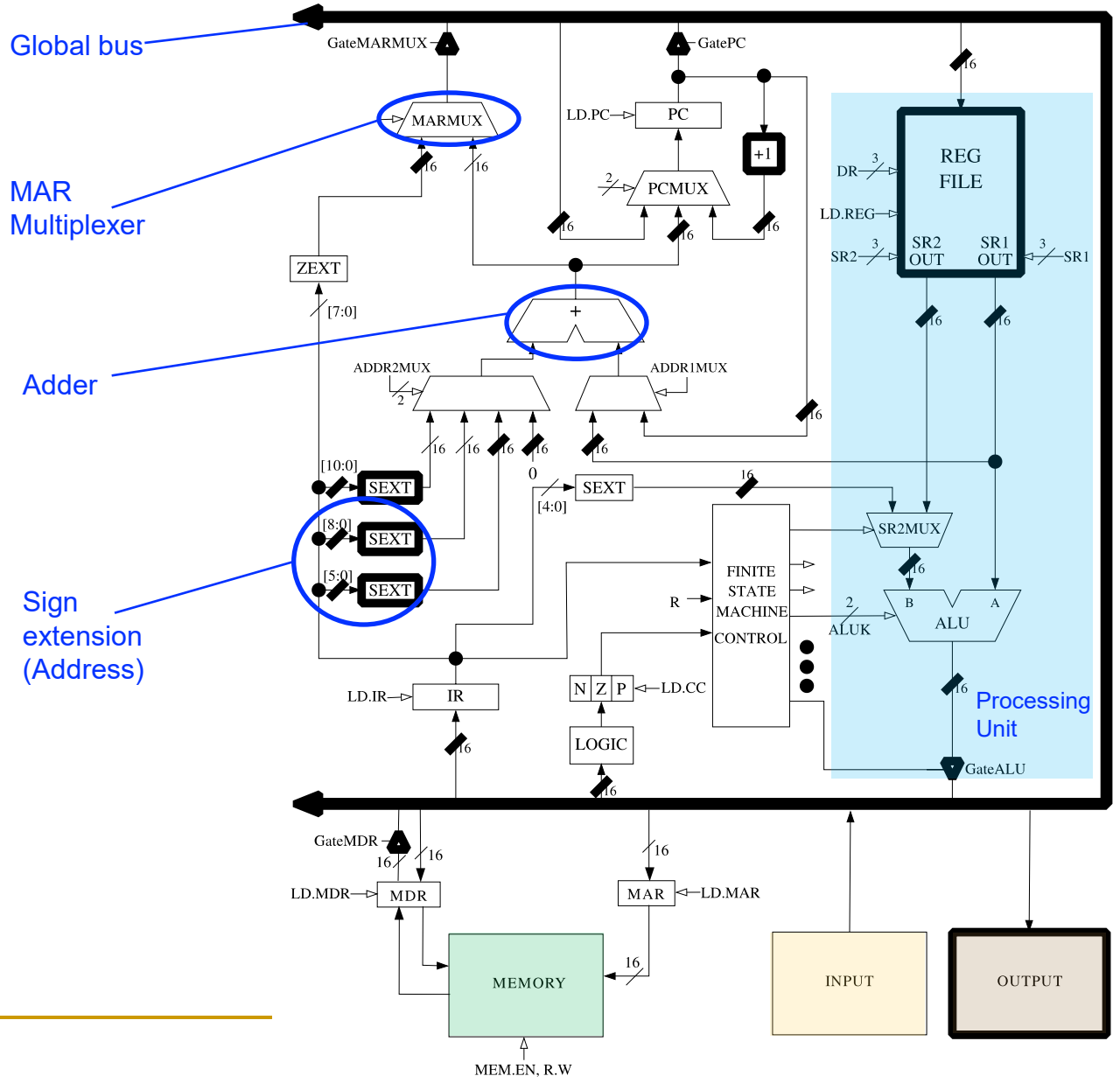
Machine Code

OP	DR	BaseR	offset6
0 1 1 0	0 0 1	0 1 0	0 1 1 1 0 1
15 12	11 9	8 6	5 0



Again, the address of the operand can be anywhere in the memory

Address Calculation in LC-3 Data Path



Base+Offset Addressing Mode in MIPS

- In MIPS, lw and sw use base+offset mode (or base addressing mode)

High-level code

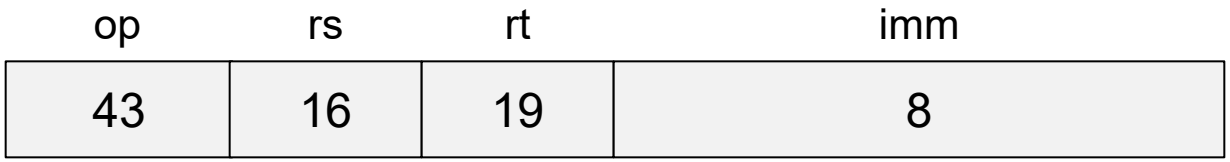
```
A[2] = a;
```

MIPS assembly

```
sw   $s3, 8($s0)
```

Memory[$s0 + 8] ← $s3

Field Values



- imm is the 16-bit offset, which is sign-extended to 32 bits

An Example Program in MIPS and LC-3

High-level code

```
a    = A[0];
c    = a + b - 5;
B[0] = c;
```

MIPS registers

```
A = $s0
b = $s2
B = $s1
```

LC-3 registers

```
A = R0
b = R2
B = R1
```

MIPS assembly

```
lw   $t0, 0($s0)
add  $t1, $t0, $s2
addi $t2, $t1, -5
sw   $t2, 0($s1)
```

LC-3 assembly

```
LDR  R5, R0, #0
ADD  R6, R5, R2
ADD  R7, R6, #-5
STR  R7, R1, #0
```

Immediate Addressing Mode (in LC-3)

- LEA (Load Effective Address)



  - OP = 1110
  - DR = destination register
  - LEA: DR ← PC† + sign-extend(PCoffset9)

What is the difference from PC-Relative addressing mode?

Answer: Instructions with PC-Relative mode load from memory, but LEA does not → Hence the name *Load Effective Address*

† This is the incremented PC

LEA in LC-3

- LEA assembly and machine code

LC-3 assembly

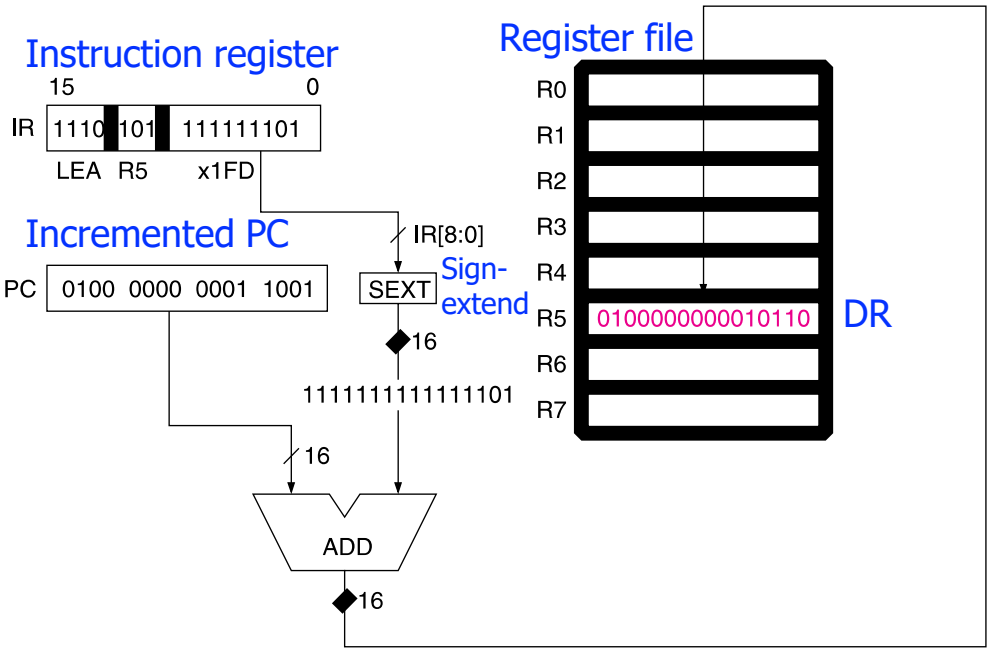
```
LEA R5, #-3
```

Field Values

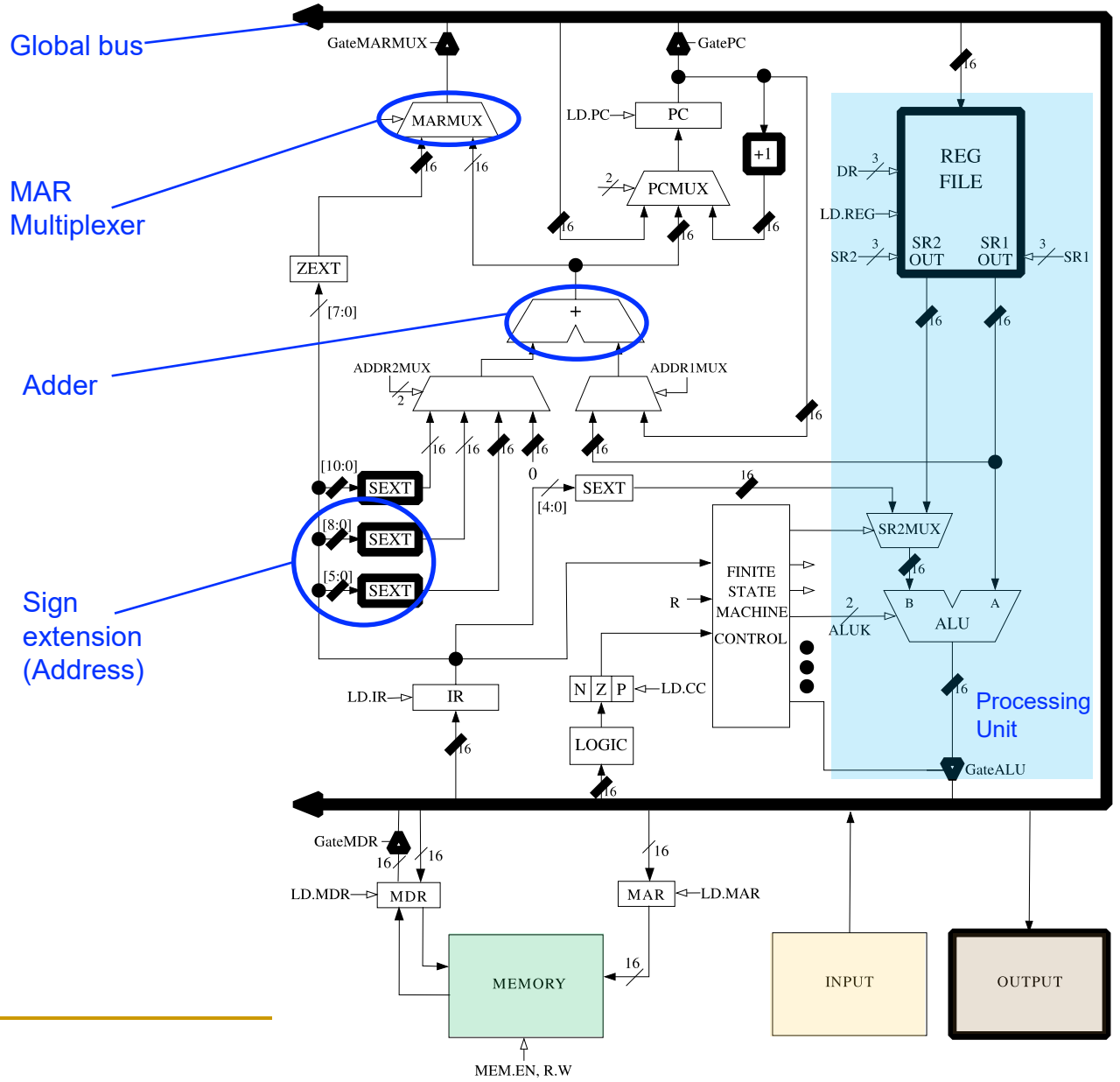
OP	DR	PCoffset9
E	5	0x1FD

Machine Code

OP	DR	PCoffset9
1 1 1 0	1 0 1	1 1 1 1 1 1 1 0 1
15 12	11 9	8 0



Address Calculation in LC-3 Data Path



Immediate Addressing Mode in MIPS

- In MIPS, lui (load upper immediate) loads a 16-bit immediate into the upper half of a register and sets the lower half to 0

- It is used to assign 32-bit constants to a register

High-level code

```
a = 0x6d5e4f3c;
```

MIPS assembly

```
# $s0 = a
lui  $s0, 0x6d5e
ori  $s0, 0x4f3c
```

Addressing Example in LC-3

- What is the final value of R3?

P&P, Chapter 5.3.5

Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
x30F6	1	1	1	0	0	0	1	1	1	1	1	1	1	1	0	1	`R1<- PC-3`
x30F7	0	0	0	1	0	1	0	0	0	1	1	0	1	1	1	0	`R2<- R1+14`
x30F8	0	0	1	1	0	1	0	1	1	1	1	1	1	0	1	1	`M[x30F4]<- R2`
x30F9	0	1	0	1	0	1	0	0	1	0	1	0	0	0	0	0	`R2<- 0`
x30FA	0	0	0	1	0	1	0	0	1	0	1	0	0	1	0	1	`R2<- R2+5`
x30FB	0	1	1	1	0	1	0	0	0	1	0	0	1	1	1	0	`M[R1+14]<- R2`
x30FC	1	0	1	0	0	1	1	1	1	1	1	1	0	1	1	1	`R3<- M[M[x30F4 ]]`

LC- ISA Instruction Encodings

Patt & Patel,
Back cover inside

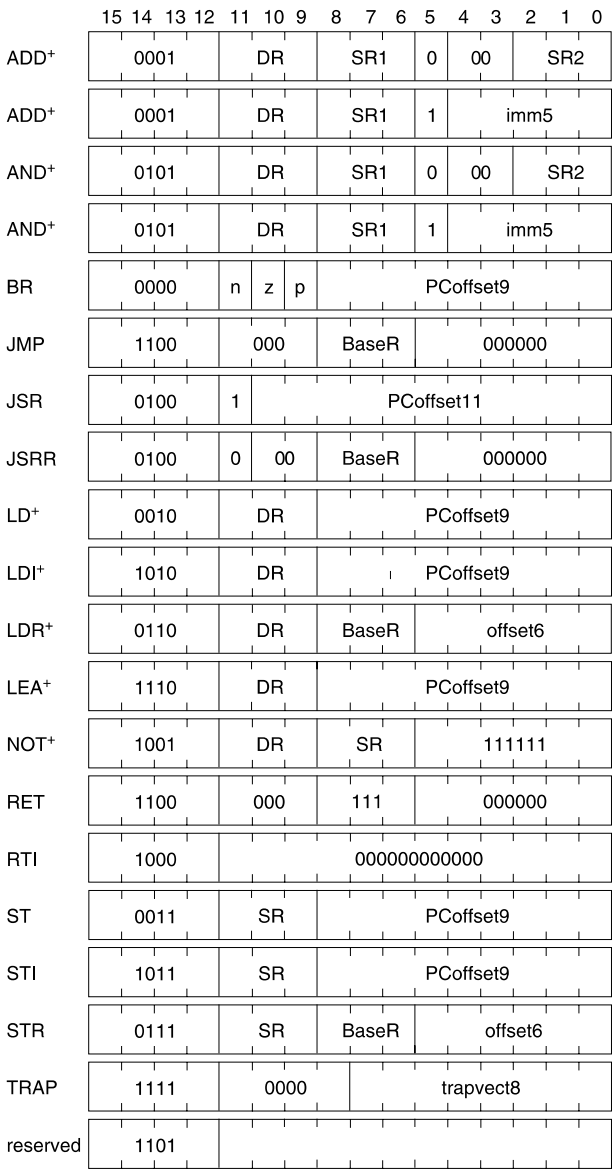
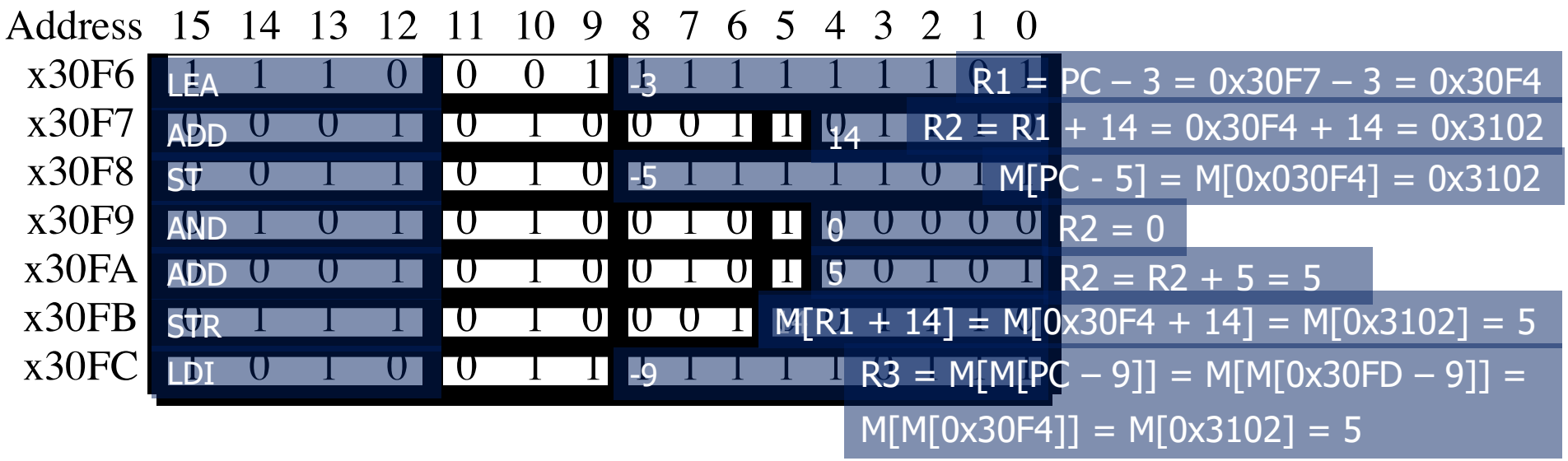


Figure 5.3 Formats of the entire LC-3 instruction set. NOTE: + indicates instructions that modify condition codes

Addressing Example in LC-3

- What is the final value of R3? [Homework]

P&P, Chapter 5.3.5



- The final value of R3 is 5

LDI: To load a constant value directly into a general-purpose register (GPR)
STR (Store Register) instruction is used to save the value of a register into memory.
LEA: calculates the memory address of an operand and stores it in a register
ST: used to write data from a register to a specific memory location

Control Flow Instructions

Control Flow Instructions

- Allow a program to execute out of sequence
- Conditional branches and unconditional jumps
  - Conditional branches are used to make decisions
    - E.g., if-else statement
  - In LC-3, three condition codes are used
  - Jumps are used to implement
    - Loops
    - Function calls
  - JMP in LC-3 and j in MIPS

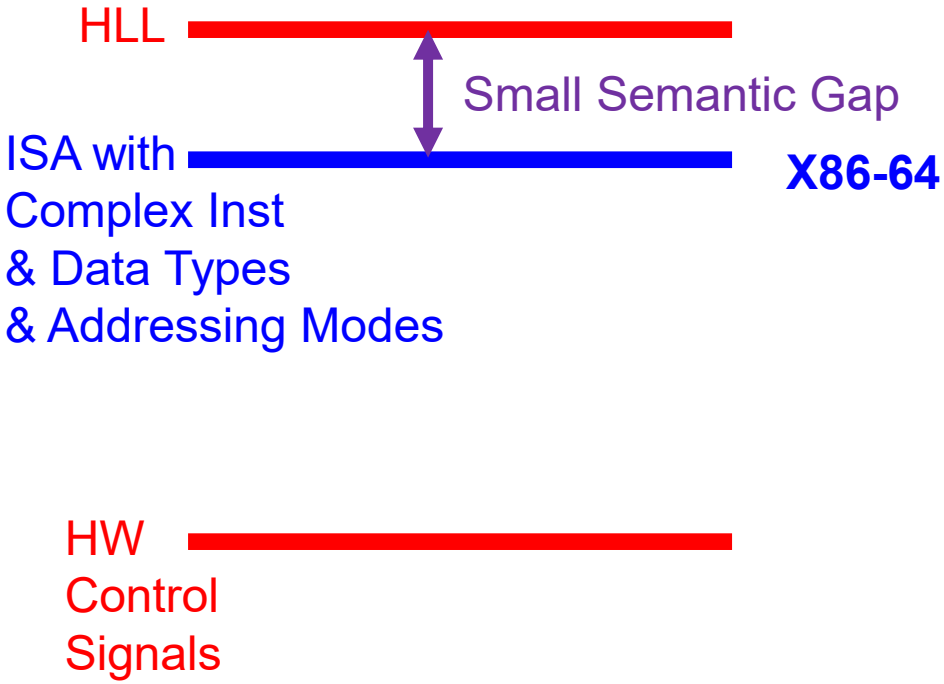
Many Different ISAs Over Decades

- x86
- PDP-x: Programmed Data Processor (PDP-11)
- VAX
- IBM 360
- CDC 6600
- SIMD ISAs: CRAY-1, Connection Machine
- VLIW ISAs: Multiflow, Cydrome, IA-64 (EPIC)
- PowerPC, POWER
- RISC ISAs: Alpha, MIPS, SPARC, ARM, RISC-V, …

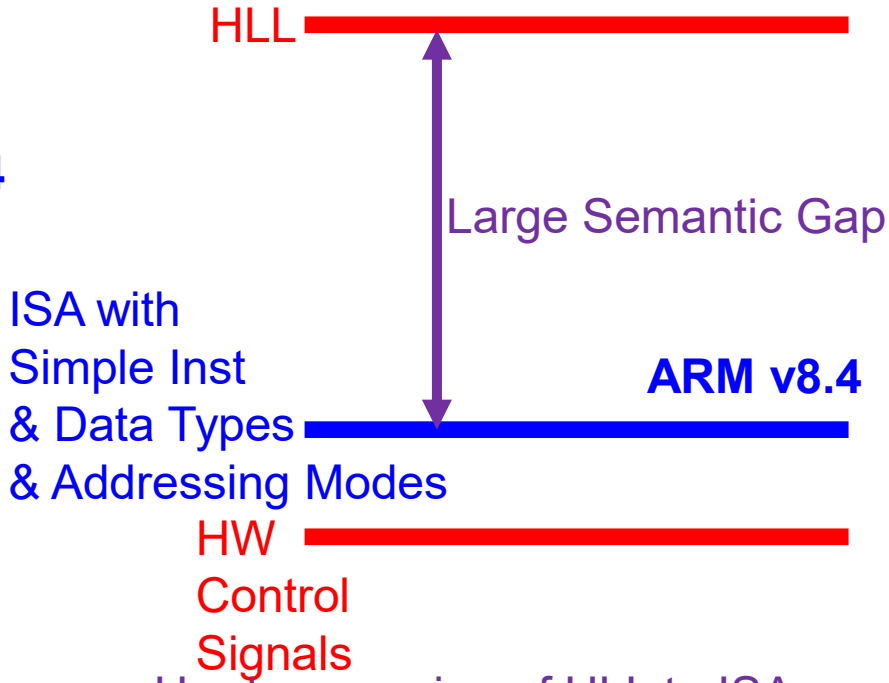
- What are the fundamental differences?
  - E.g., how instructions are specified and what they do
  - E.g., how complex are instructions, data types, addr. modes

Semantic Gap

- How close instructions & data types & addressing modes are to high-level language (HLL)



Easier mapping of HLL to ISA
Less work for software designer
More work for hardware designer
Optimization burden on HW



Harder mapping of HLL to ISA
More work for software designer
Less work for hardware designer
Optimization burden on SW

The Dataflow Execution Model of a Computer

The Dataflow Model (of a Computer)

- Von Neumann model: An instruction is fetched and executed in control flow order
  - As specified by the program counter (instruction pointer)
  - Sequential unless explicit control flow instruction

- Dataflow model: An instruction is fetched and executed in data flow order
  - i.e., when its operands are ready
  - i.e., there is no program counter (instruction pointer)
  - Instruction ordering specified by data flow dependence
    - Each instruction specifies "who" should receive the result
    - An instruction can "fire" whenever all operands are received
  - Potentially many instructions can execute at the same time
    - Inherently more parallel

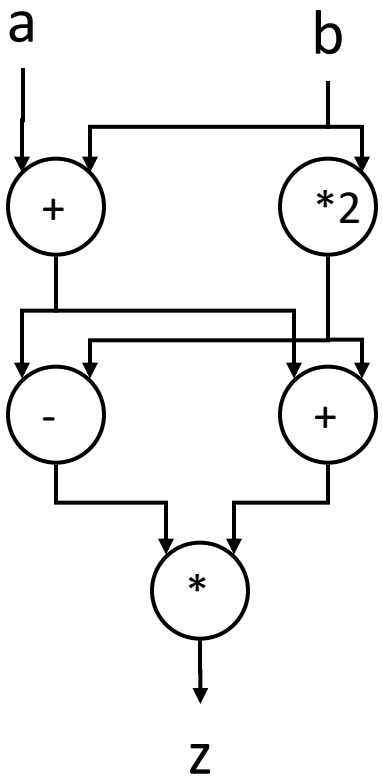
Von Neumann vs. Dataflow

- Consider a Von Neumann program
  - What is the significance of the program order?
  - What is the significance of the storage locations?

```
v = a + b;
w = b * 2;
x = v - w
y = v + w
z = x * y
```

Sequential

a, b are the only inputs
z is the only output



Dataflow

A data flow execution model describes how data moves and is processed within a system, representing operations as nodes and data paths as edges in a graph.

Which model is more natural to you as a programmer?

Von Neumann vs. Dataflow

Feature	Von Neumann	Data Flow
Execution Order	Sequential	Data-driven (parallel)
Memory Access	Shared (instructions and data)	Separate (instructions and data as tokens)
Control	Centralized	Decentralized
Performance Bottleneck	Von Neumann bottleneck	Potential for high parallelism
Complexity	Simpler	More complex
Adoption	Ubiquitous	Specialized applications

Existing architectural Classification

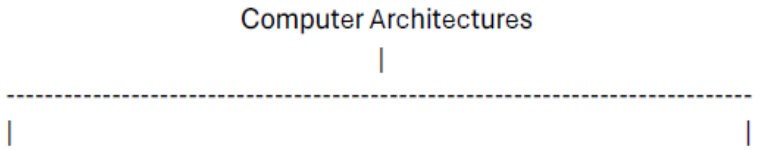
2.1. Flynn's Classification:

Based on processing (the number of instruction streams and data streams are processed simultaneously) computer architectures are classified in four categories, SISD, SIMD, MISD and MIMD. Their characteristic comparison is as followed.

Architecture	Instruction Stream	Data Stream	Examples	Characteristics
SISD	Single	Single	Traditional CPU	Sequential execution
SIMD	Single	Multiple	GPU, Vector Processor, SPAR-2	Data parallelism
MISD	Multiple	Single	Fault-tolerant systems	Rarely used
MIMD	Multiple	Multiple	Multicore CPUs, Clusters	Task parallelism

2.2. Memory organization Classification:

Based on memory organization and execution model, design architectures are classified in two categories, Von Neumann vs. Non-Von Neumann architecture.



Von Neumann Architecture

CPU

Shared Memory

Shared Bus

Sequential Fetch-Decode-Execute

Non-Von Neumann Architectures

Harvard Architecture

Dataflow Architecture

Processing-In-Memory (PIM)

Neuromorphic Computing

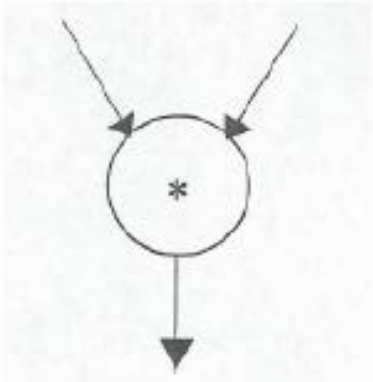
Quantum Computing

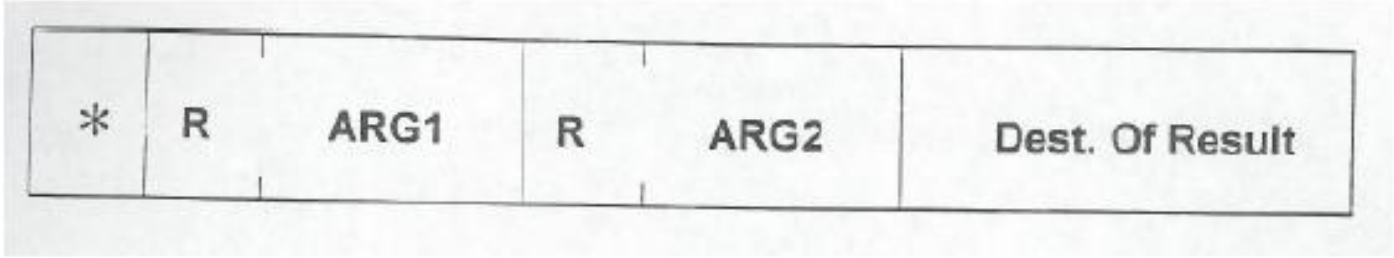
Cellular/Other Specialized Architectures

More on Dataflow

- In a dataflow machine, a program consists of dataflow nodes
  - A dataflow node fires (fetched and executed) when all it inputs are ready
    - i.e. when all inputs have tokens

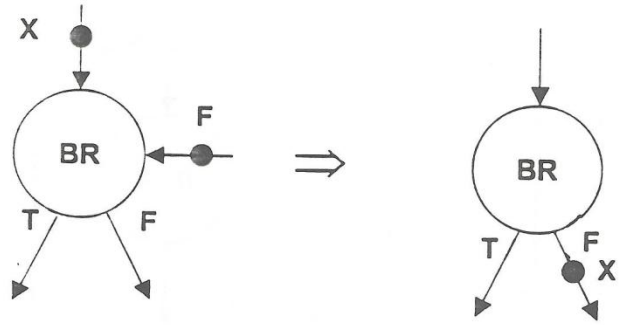
- Dataflow node and its ISA representation



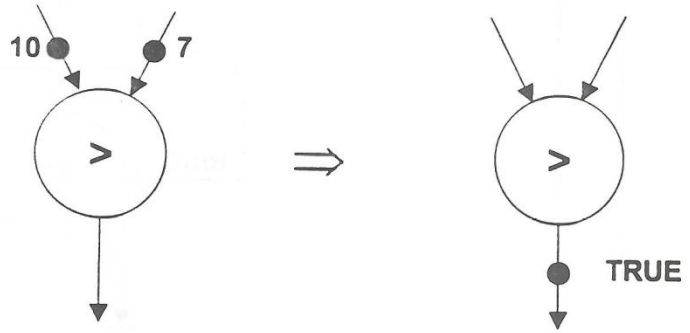


Example Dataflow Nodes

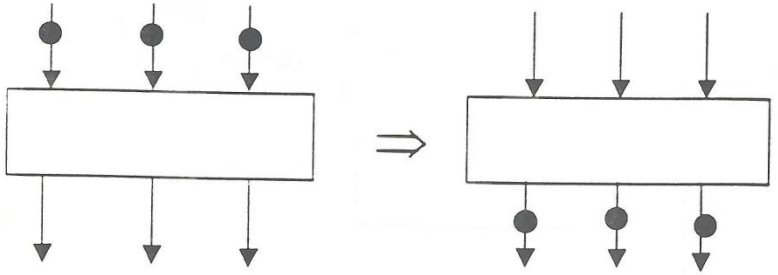
*Conditional



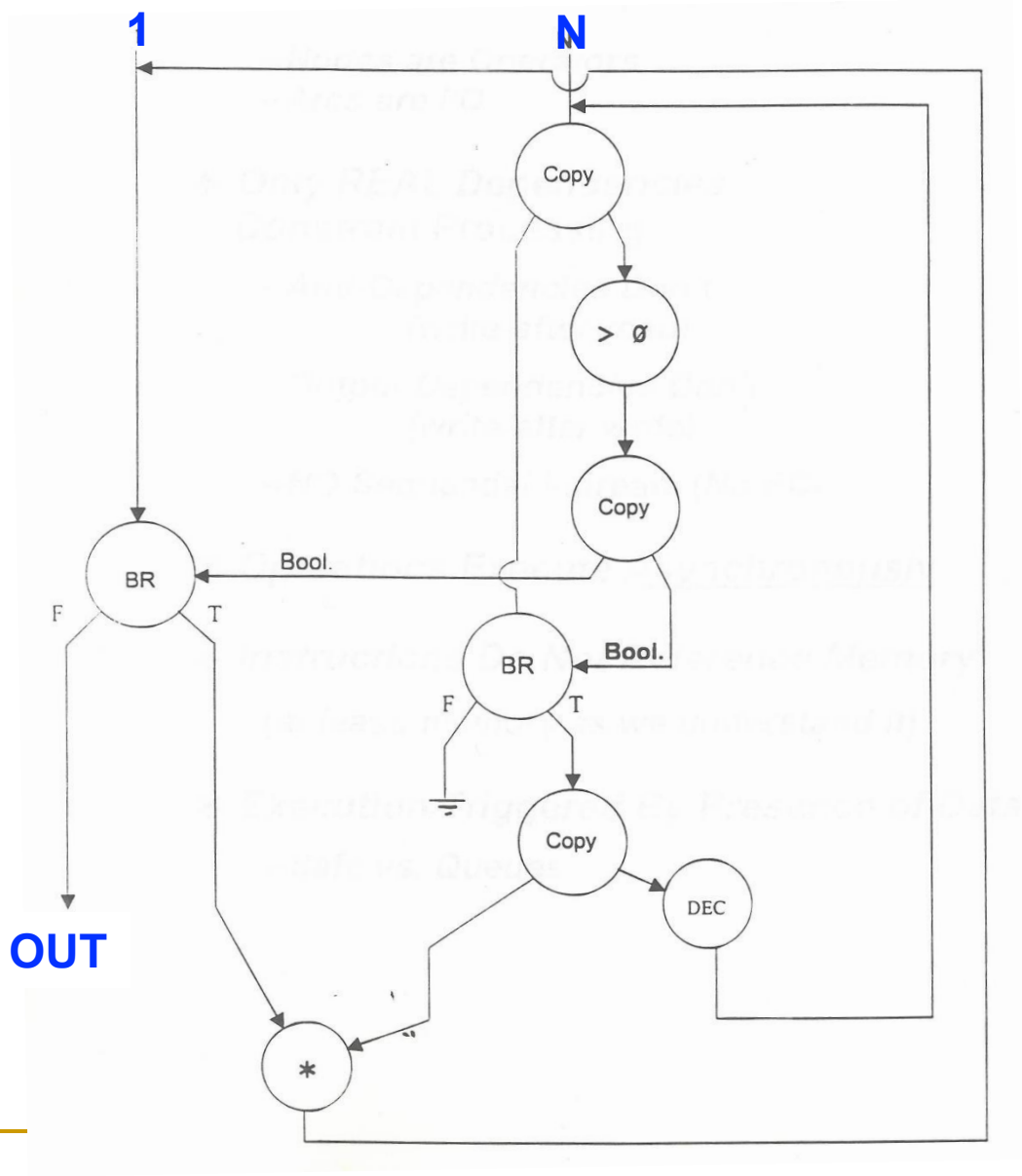
*Relational



*Barrier Synch



A Simple Example Dataflow Program



N is a non-negative integer

What is the value of OUT?

Solution

The flowchart is implementing a **factorial operation**.

Operation

It repeatedly:

1. Takes the input **N**.
2. Checks whether **N > 0**.
3. If **N > 0**, it:
   - uses the current value of `N` for multiplication,
   - decrements `N` using `DEC`,
   - repeats the process.
4. When `N` becomes 0, the branch terminates and produces the result.

Therefore, the output is:

$$\boxed{N! = N \times (N-1) \times (N-2) \times \cdots \times 1}$$

Example

If the input is:

$$N = 5$$

the computation is:

$$5 \times 4 \times 3 \times 2 \times 1 = \boxed{120}$$

So the **output of the flowchart is** $N!$.

For N = 0, the output would be:

$$\boxed{1}$$

because $0! = 1$.

ISA vs. Microarchitecture Level Tradeoff

- A similar tradeoff (control vs. data-driven execution) can be made at the microarchitecture level

- ISA: Specifies how the **programmer sees** the instructions to be executed
  - Programmer sees a sequential, control-flow execution order vs.
  - Programmer sees a dataflow execution order

- Microarchitecture: How the **underlying implementation actually executes** instructions
  - Microarchitecture can execute instructions in any order as long as it obeys the semantics specified by the ISA when making the instruction results visible to software
    - Programmer should see the order specified by the ISA

The von Neumann Model

- All major *instruction set architectures* today use this model
  - x86, ARM, MIPS, SPARC, Alpha, POWER, RISC-V, …
- Underneath (at the microarchitecture level), the execution model of almost all *implementations (or, microarchitectures)* is very different
  - Pipelined instruction execution: *Intel 80486 uarch*
  - Multiple instructions at a time: *Intel Pentium uarch*
  - Out-of-order execution: *Intel Pentium Pro uarch*
  - Separate instruction and data caches
- But, what happens underneath that is ***not* consistent** with the von Neumann model is ***not* exposed** to software
  - Difference between ISA and microarchitecture

Microarchitecture

- A specific **implementation** of the ISA



- There can be many implementations of the same ISA
  - **MIPS** R2000, R3000, R4000, R6000, R8000, R10000, …
  - **x86**: Intel 80486, Pentium, Pentium Pro, Pentium 4, Kaby Lake, Coffee Lake, Comet Lake, Ice Lake, Golden Cove, Sapphire Rapids, …, AMD K5, K7, K9, Bulldozer, BobCat, Ryzen X, …
  - **POWER** 4, 5, 6, 7, 8, 9, 10 (IBM), …, **PowerPC** 604, 605, 620, …
  - **ARM** Cortex-M*, ARM Cortex-A*, NVIDIA Denver, Apple A*, M1, …
  - **Alpha** 21064, 21164, 21264, 21364, …
  - **RISC-V** …
  - …

ISA vs. Microarchitecture



- What is part of ISA vs. Uarch?
  - Gas pedal: interface for "acceleration"
  - Internals of the engine: implement "acceleration"

- Implementation (uarch) can be various as long as it satisfies the specification (ISA)
  - Add instruction vs. Adder implementation
    - Bit serial, ripple carry, carry lookahead adders are all part of microarchitecture **(see H&H Chapter 5.2.1)**
  - x86 ISA has many implementations:
    - Intel 80486, Pentium, Pentium Pro, Pentium 4, Kaby Lake, Coffee Lake, Comet Lake, Ice Lake, Golden Cover, Sapphire Rapids, …, AMD K5, K7, K9, Bulldozer, BobCat, Ryzen X, …

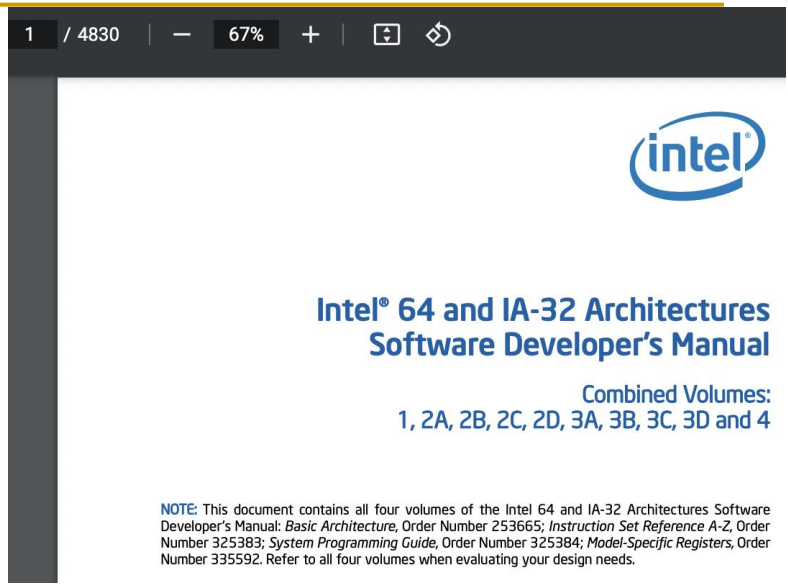
- Microarchitecture usually changes faster than ISA
  - Few ISAs (x86, ARM, SPARC, MIPS, Alpha, RISC-V) but many uarchs

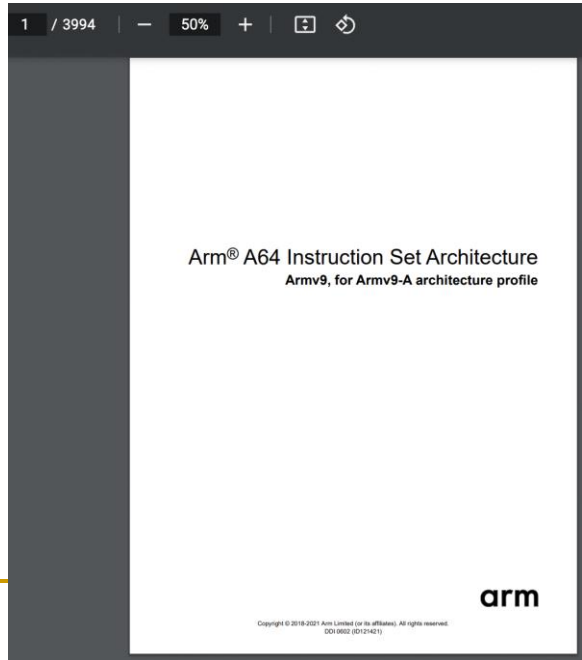
Intel's "Nehalem", "Sandy Bridge", and "Ivy Bridge" microarchitectures, which all implement the x86-64 ISA.

https://www.vox.com/2015/7/1/8877583/two-foot-driving-pedal-error

ISA: What Does It Specify?

- Instructions
  - Opcodes, Addressing Modes, Data Types
  - Instruction Types and Formats
  - Registers, Condition Codes
- Memory
  - Address space, Addressability, Alignment
  - Virtual memory management
- Call, Interrupt/Exception Handling
- Access Control, Priority/Privilege
- I/O: memory-mapped vs. instructions
- Task/thread Management
- Power & Thermal Management
- Multithreading & Multiprocessor support
- …





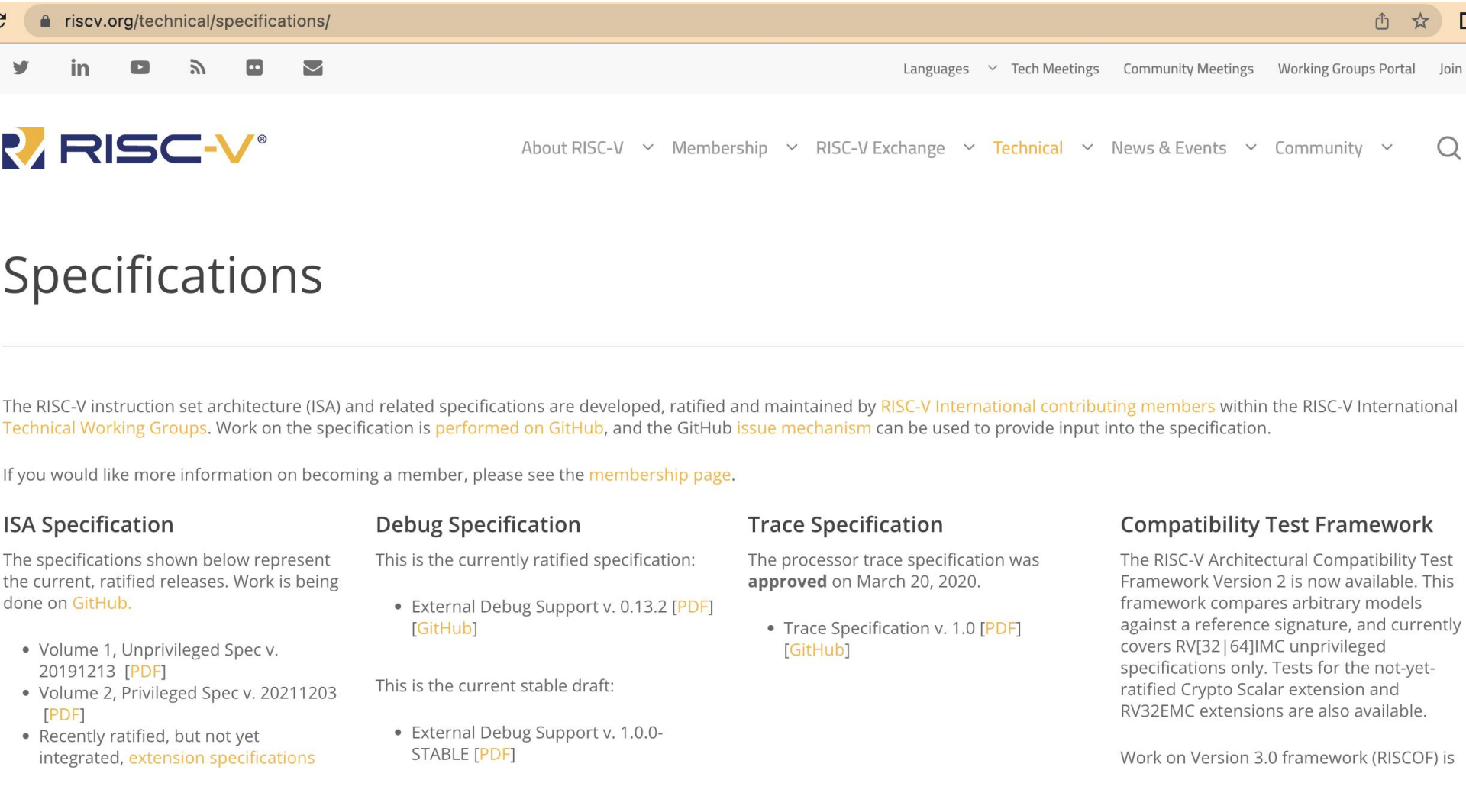
ISA Manuals: Some Good Bedtime Reading

Combined Volume Set of Intel® 64 and IA-32 Architectures Software Developer's Manuals

Document	Description
Intel® 64 and IA-32 Architectures Software Developer's Manual Combined Volumes: 1, 2A, 2B, 2C, 2D, 3A, 3B, 3C, 3D, and 4	This document contains the following: **Volume 1**: Describes the architecture and programming environment of processors supporting IA-32 and Intel® 64 architectures. **Volume 2**: Includes the full instruction set reference, A-Z. Describes the format of the instruction and provides reference pages for instructions. **Volume 3**: Includes the full system programming guide, parts 1, 2, 3, and 4. Describes the operating-system support environment of Intel® 64 and IA-32 architectures, including: memory management, protection, task management, interrupt and exception handling, multi-processor support, thermal and power management features, debugging, performance monitoring, system management mode, virtual machine extensions (VMX) instructions, Intel® Virtualization Technology (Intel® VT), and Intel® Software Guard Extensions (Intel® SGX). NOTE: Performance monitoring events can be found here: https://perfmon-events.intel.com/ **Volume 4**: Describes the model-specific registers of processors supporting IA-32 and Intel® 64 architectures.
Intel® 64 and IA-32 Architectures Software Developer's Manual Documentation Changes	Describes bug fixes made to the Intel® 64 and IA-32 architectures software developer's manual between versions. NOTE: This change document applies to all Intel® 64 and IA-32 architectures software developer's manual sets (combined volume set, 4 volume set, and 10 volume set).

https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html

ISA Manuals: Some Good Bedtime Reading



https://riscv.org/technical/specifications/